

Lab Workbook — Manual Testing and Defect Report

Project: MindBridge Course Registration platform — new checkout flow

Your name

Student

Date

8 Aug 2026

Jira project name

Project

Complete all three tasks below with both text and screenshots. When finished, export this document as a single PDF — that PDF is your submission for the graded assignment.

Task 1: Exploratory testing session summary

Summarize your 15-minute exploratory session. List the heuristics you applied and the bugs you discovered.

Session summary (what you tested and how):

I ran a 15-minute time-boxed exploratory session against the MindBridge checkout flow running locally at `http://localhost:3000`. The charter was to explore the new checkout flow with a focus on input validation and transaction state changes. Rather than only walking the happy path, I probed each step of the purchase journey: browsing the Catalogue, opening a course detail page, adding a course to the cart, adjusting quantity in the cart, and completing the payment form. Chrome DevTools was open throughout with the Network tab recording and “Preserve log” enabled, so I could read the HTTP status code and response body for every request instead of relying on what the page displayed on screen.

Environment: Google Chrome version 143 on the Linux lab desktop, application served at localhost:3000. The session produced one confirmed defect — a server-side crash triggered by unsanitised payment input — together with a supporting client-side validation weakness that allows the bad input through. I also recorded one negative result: repeated rapid submissions of “Place order” did not produce duplicate orders in my attempts, so I could not confirm a transaction-state defect within the time box. I have noted this as covered-but-not-confirmed rather than omitting it, since a heuristic that finds nothing is still evidence of coverage.

Heuristics applied (name at least two):

- Unexpected Characters — entered HTML special characters into the payment fields to see whether input that does not match the expected format is rejected safely rather than passed through to the server.
- Rapid Input — selected “Place order” and “Add to cart” repeatedly in quick succession to see whether the application guards against duplicate submissions while a request is still in flight.
- Boundary / Goldilocks — submitted empty, too-short and malformed values in the card number, expiry, CVV and zip fields to find where validation starts and stops.

Bugs discovered (one line each, naming the feature area and the observed failure):

- Checkout / payment processing — a card number containing HTML special characters causes POST /api/checkout to return HTTP 500 “Internal server error”; no order is created and the user is offered no recovery path.
- Checkout / client-side validation — the card number check counts digits only and does not reject non-digit characters, so “4242 4242 4242 4242</script>” passes validation in the browser and is forwarded to the server unsanitised.

Task 2: Your Jira bug report

Copy the text from each field of the Jira ticket you filed for the most critical defect you found, then insert a screenshot of the complete ticket below the text.

Summary

Checkout: “Place order” returns HTTP 500 when the card number field contains HTML special characters

Steps to Reproduce

1. Log in as an existing user, open the Catalogue and add one course to the cart (I used “distributed QA Strategy blockchains”, \$133.81). Open the cart and select Checkout.
2. Open Chrome DevTools, select the Network tab and tick “Preserve log” so the requests survive the page transition.
3. In the Payment (mock) section enter exactly — Card number: 4242 4242 4242 4242</script> | Expiry: 12/29 | CVV: 123 | Zip code: 94105.
4. Select “Place order” once, then observe both the Network tab and the checkout page.

Expected Result

The card number is rejected by validation with an inline field-level error message, in the same way that other malformed card numbers are rejected (for example, a short number returns “Card number must be 13–19 digits.”). The cart contents are preserved and the user is told how to correct the input.

Actual Result

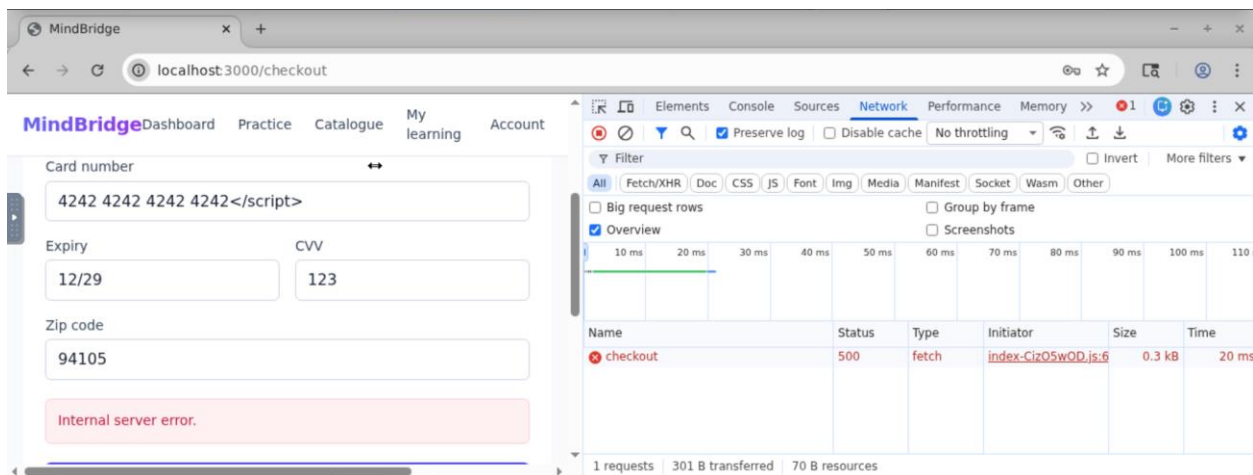
Client-side validation passes and the request is sent. POST /api/checkout returns HTTP 500 (0.3 kB response, 20 ms) and the checkout page displays the message “Internal server error.” No order is created, no confirmation page is shown, and no field-level guidance is

given, so the user has no way to recover. Contributing factor observed: the client-side card number check counts digits only and does not reject non-digit characters, so a valid 16-digit number with an appended tag satisfies the browser check and reaches the server unsanitised.

Severity

Critical — an unhandled server error on the payment endpoint, reachable by any user through ordinary form input, that blocks the purchase and returns a generic error offering no recovery path.

Visual evidence:



Task 3: Defect recovery — the rejected ticket

Describe each specific change you made to the rejected "Cannot Reproduce" ticket, and explain why that change makes the defect easier for a developer to reproduce. Then insert a screenshot of the updated ticket showing the new details and the **In Progress** status.

Change I made to the ticket

Why this helps a developer reproduce the defect

Change 1: Rewrote the summary from "Checkout is broken" to "Checkout: 'Place order' returns HTTP 500 when the card number field contains HTML special characters".

Why this helps: The original summary names no component, no trigger and no observable failure, so it cannot be searched, triaged or duplicate-checked. The new summary tells a developer the affected area, the action that triggers the fault and the symptom before they open the ticket.

Change 2: Added numbered Steps to Reproduce that begin from a defined starting state (logged-in user, one named course in the cart).

Why this helps: The original ticket contained no steps at all, so the developer improvised their own path through checkout and hit the working case. Specifying the starting state removes the guesswork about cart contents and account state that made their attempt differ from mine.

Change 3: Recorded the literal values entered in each payment field instead of describing them, including the exact card number string 4242 4242 4242</script>.

Why this helps: The developer explicitly asked what was entered in the payment fields. An exact string lets them reproduce the input character for character; “the payment fields” gave them nothing to type. This defect only fires when the input contains ‘<’ or ‘>’, so the precise characters are the whole trigger.

Change 4: Split the single “didn’t work right” statement into separate Expected Result and Actual Result fields.

Why this helps: Collapsing the two together left the developer unable to tell whether the flow failed or merely behaved unexpectedly. Stating them separately makes the deviation explicit and gives a clear pass/fail condition for the fix.

Change 5: Added the concrete failure signature: HTTP 500 on POST /api/checkout, 0.3 kB response, and the on-screen text “Internal server error.”

Why this helps: A status code and an exact error string give the developer something to assert against, so they know with certainty

whether they have reproduced the defect. “Didn’t work right” gives them no such test.

Change 6: Added an Environment section recording the browser and version, the operating system and the application URL.

Why this helps: The developer’s stated reason for rejection was that it worked on their machine, which implies a suspected environment difference. Recording the exact environment either rules that out or gives them the specific configuration to test on.

Change 7: Added the DevTools instruction to enable “Preserve log” before submitting, and attached a screenshot of the Network tab showing the 500 response.

Why this helps: Without “Preserve log” the page transition clears the request list, so a developer following the steps could reproduce the fault and still not see the evidence. The screenshot also proves the failure is real on a ticket that has already been rejected once.

Change 8: Set the Priority field, which was empty, and removed the now-inaccurate “cannot-reproduce” label before transitioning the ticket to In Progress.

Why this helps: An unset priority carries no urgency signal into triage, so a critical payment-path crash could sit alongside cosmetic issues. Leaving the stale label attached would keep the ticket in filters and reports for defects that cannot be reproduced, which is no longer true.

Visual evidence:

The image shows a screenshot of a web application dashboard on the left and a browser DevTools network tab on the right.

Dashboard (MindBridge):

- Navigation: Dashboard, Practice, Catalogue, My learning, Account
- User: world-class Performance Testing schemas, Enrolled - active
- Button: Back to catalogue

DevTools Network Tab:

- Filter: All, Fetch/XHR, Doc, CSS, JS, Font, Img, Media, Manifest, Socket, Wasm, Other
- Checkboxes: ☐ Big request rows, ☒ Overview, ☐ Group by frame, ☐ Screenshots
- Timeline: 100,000 ms to 1,000,000 ms
- Table:

Name	Status	Type	Initiator	Size	Time
checkout	500	fetch	index-CizO5wOD.js:6	0.3 kB	20 r
checkout	201	fetch	index-CizO5wOD.js:6	0.6 kB	72 r
favicon.ico	304	text/html	Other	0.2 kB	18 r